

IBM SkillsBuild Bob Hackathon Showcase – Gujarat, 2026

Project Title – FLOODGUARD AI

Domain of Project – Urban Infrastructure & Disaster Management

Student Name	Recent Passport Photo	Email ID	Mobile number [WhatsApp]
Prince Jain		princejain566@gmail.com	9753995851
Alka Raikwar		alkaraikbar755@gamil.com	6207020293
Aniket Jaiswal		officialaniket56@gmail.com	6267168690

IBM SkillsBuild Bob Hackathon Showcase Gujarat, 2026

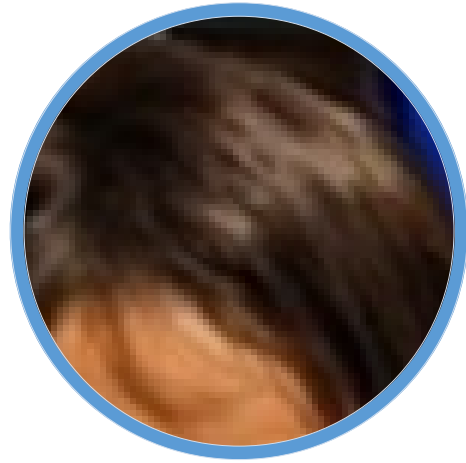


PARUL UNIVERSITY

FloodGuard AI — Agentic AI-Powered Urban Flood Intelligence & Disaster Response System



Team Leader



Team member 1



Team member 2

FloodGuard AI is an Agentic AI-powered urban flood intelligence platform designed for Ahmedabad and Surat. It combines live weather evidence, ML-based flood-risk prediction, citizen reports, geospatial intelligence, and multi-agent reasoning to identify high-risk zones and generate explainable, prioritized response recommendations.

Methodology:

FloodGuard AI uses a multi-agent decision intelligence workflow. Live weather and application data are analysed by specialized agents for flood risk, drainage, citizen reports, and response coordination. Evidence Fusion combines these signals to prioritize risky zones and explain Why This Zone and Why Now. AI-generated response recommendations are then reviewed through Human-in-the-Loop approval.

IBM Tools Used:

- IBM Bob: AI-assisted application development
- IBM Granite: AI understanding, explanations & recommendations
- IBM Watsonx: Granite integration & AI workflow
- IBM Cloud: Cloud infrastructure & IBM services
- Agentic AI: Multi-agent decision workflow
- Machine Learning: Flood-risk prediction from key flood data

Conclusion and Demonstration:

FloodGuard AI transforms urban flood data into actionable decision intelligence.

The demonstration shows the complete flow:

Weather & Citizen Reports → ML Risk Prediction → Multi-Agent Analysis → Evidence Fusion → Zone Prioritization → AI Recommendation → Human Approval → Audit Trail

This enables authorities to move from reactive flood response to explainable, prioritized and data-driven decision support.

“FloodGuard AI doesn't just predict flooding — it converts evidence into prioritized action.”

FloodGuard AI: Agentic Urban Flood Intelligence

- **What is the project?**

An Agentic AI platform designed to help authorities monitor urban flood risk and coordinate faster, data-driven responses across Ahmedabad and Surat.

- **Core Function:**

It combines live weather, ML flood-risk prediction, citizen reports, drainage data and multi-agent analysis to identify and prioritize risky zones.

- **Key Highlights:**

- Multi-Agent Flood Intelligence
- Live Weather + ML Risk Prediction
- Citizen Flood Reporting & Geospatial Mapping
- Evidence Fusion & Explainable AI
- AI Response Recommendations with Human Approval



Urban Flooding Is Predictable. The Response Is Not.

The challenge isn't just flooding — it's fragmented information and delayed response. Ahmedabad-Surat ka context already challenge mein hai, isliye sentence short rahega..



HEAVY RAINFALL

Intense rain overwhelms urban drainage, causing rapid waterlogging.



DRAINAGE FAILURE

Blocked or inadequate drains create localized flooding hotspots.



FRAGMENTED DATA

Rainfall, drainage and citizen reports remain disconnected.



DELAYED RESPONSE

Manual correlation delays decisions until damage has already spread.

How can fragmented flood information be transformed into fast, localized, actionable decisions — before the damage is done?

Fragmented signals → Delayed decisions → Greater impact

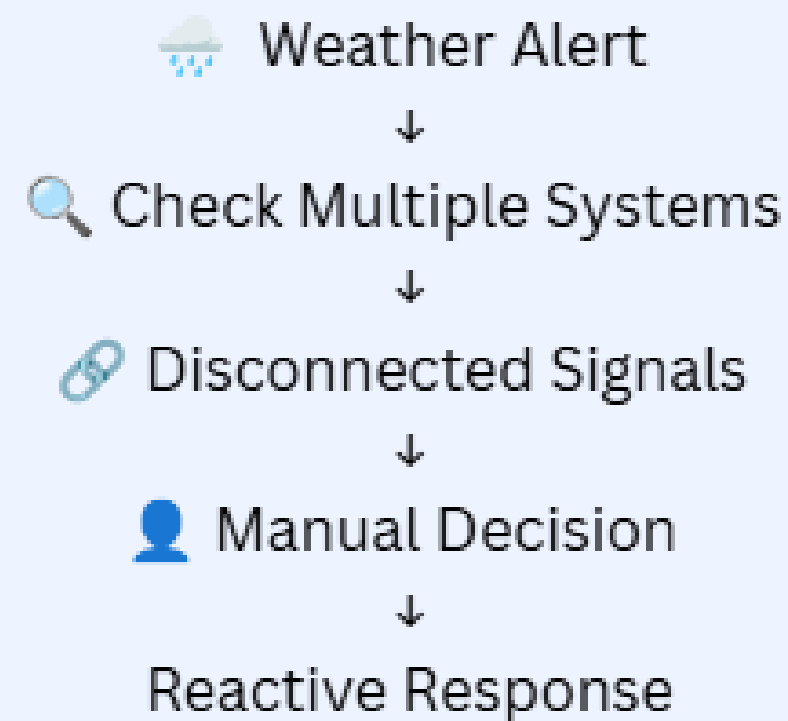
FloodGuard AI bridges detection and coordinated civic response.

Gujarat — urban flooding annually during monsoon season • Civic response typically delayed • Drainage records not correlated with weather data

Knowing a Flood Is Coming Does Not Tell You What to Do

Prediction identifies risk. Decision intelligence turns risk into action.

● PREDICTION-ONLY.



VS

● FLOODGUARD AI

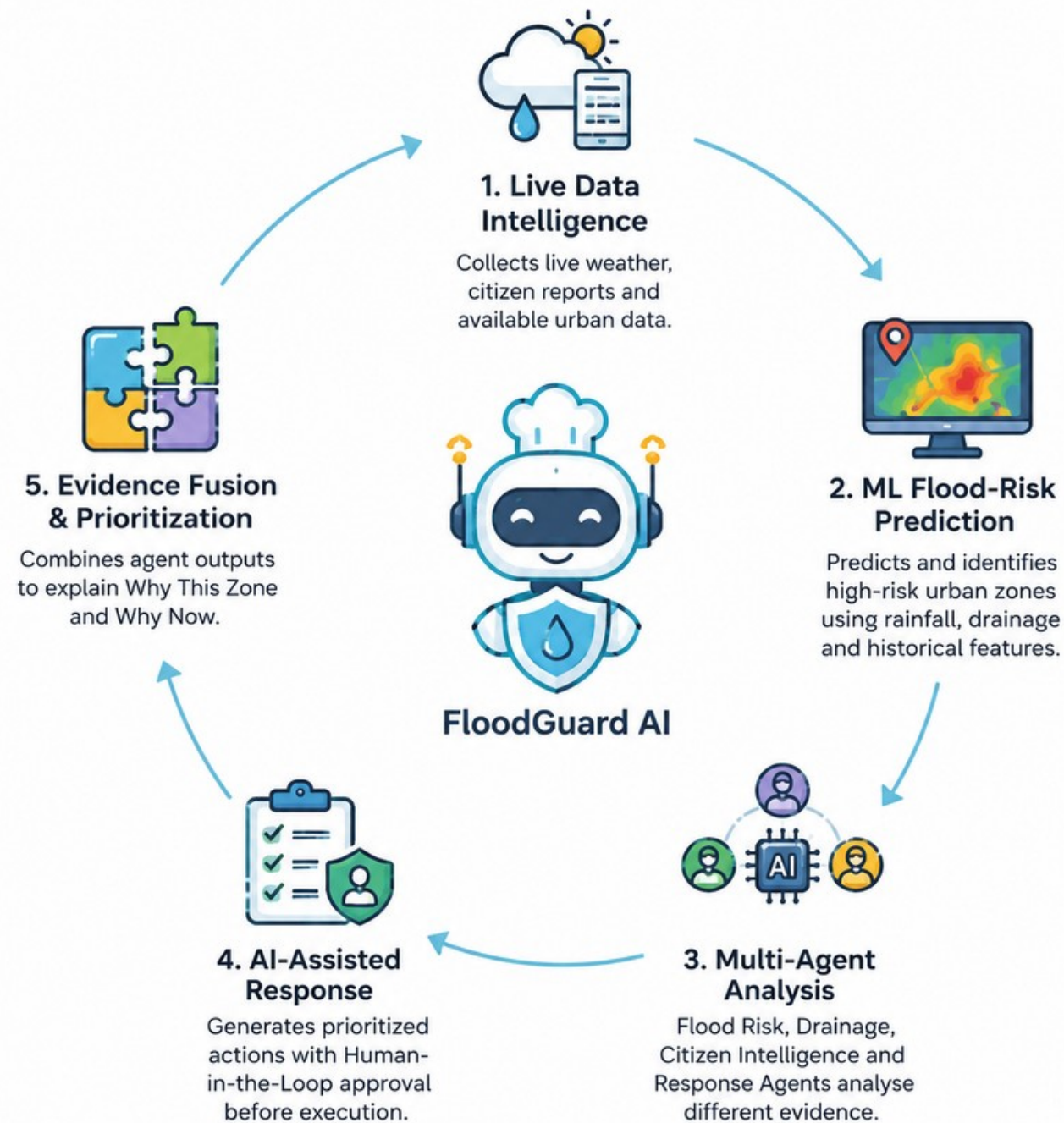
ML predicts flood risk per zone from rainfall, history, and drainage features.



The difference is not prediction — it's the decision layer.

Proposed Solution — FloodGuard AI

FloodGuard AI: 5-Step Solution



- **Live Data Intelligence:** Collects live weather, citizen reports and available urban data.
- **ML Flood-Risk Prediction:** Predicts and identifies high-risk urban zones using rainfall, drainage and historical features.
- **Multi-Agent Analysis:** Flood Risk, Drainage, Citizen Intelligence and Response Agents analyse different evidence.
- **AI-Assisted Response:** Generates prioritized actions with Human-in-the-Loop approval before execution.
- **Evidence Fusion & Prioritization:** Combines agent outputs to explain Why This Zone and Why Now.

“From flood prediction to prioritized action.”

Need of Project – FloodGuard AI

1 Reactive Flood Response



Late Detection & Delayed Civic Action

- Flood response often begins after conditions become critical.

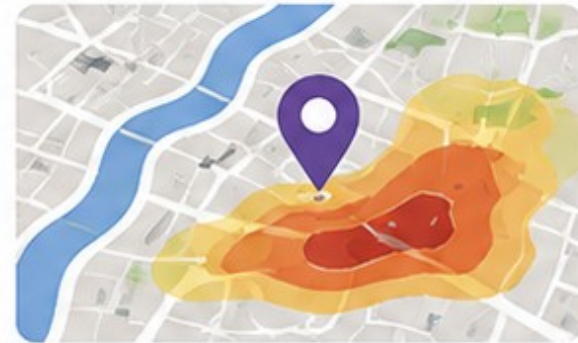
2 Fragmented Urban Data



Weather + Drainage + Citizen Reports

- Critical information exists across different sources.

3 Hyper-Local Risk Visibility



Which Zone? Why Now?

- Authorities need localized flood-risk intelligence instead of only city-level information.



4 Drainage Maintenance Priority



Identify Vulnerable Drains & Prioritize Work

- Drainage information helps identify where maintenance attention is needed.

5 Decision Support & Trust



Explainable AI + Human Approval

- AI recommendations are grounded in available evidence and reviewed by humans before critical action.

Two Distinct AI Roles — One Coordinated System



MACHINE LEARNING

FLOOD RISK PREDICTION



What it does:

Computes a flood risk score per zone from historical and real-time features.



Key inputs:

- Rainfall intensity
- Historical flood frequency for the zone
- Drainage blockage status
- Combined zone-level features



Output:

A zone risk classification:
HIGH / MEDIUM / MONITOR



Characteristic:

Statistical. Feature-driven. Quantitative.
Does not understand context or generate language.



AGENTIC AI

IBM GRANITE



What it does:

Understands context, fuses evidence from multiple agents, explains risk, generates grounded recommendations.



Key capabilities:

- Reads live application data as context (Grounded AI)
- Coordinates four specialized agents
- Synthesizes ML output + agent insights
- Explains WHY a zone is at risk
- Generates explainable, actionable response plans



Output:

A human-readable, evidence-based recommendation for civic response.



Characteristic:

Contextual. Language-capable. Multi-source. Explainable.

Four Agents — One Coordinated Intelligence

Each specialized agent analyzes one dimension of flood risk. IBM Bob orchestrates the agents, while IBM Granite synthesizes their outputs.



IBM BOB ORCHESTRATES → FOUR SPECIALIZED AGENTS → IBM GRANITE SYNTHESIZES → UNIFIED ZONE INTELLIGENCE.

These are not independent chatbots. They are coordinated intelligence components that contribute to a single, unified civic decision.

Combining All Evidence Into Prioritized Zone Intelligence

FloodGuard AI combines ML risk, drainage conditions, citizen reports and agent analysis into one zone-level intelligence picture.



IBM GRANITE — EVIDENCE FUSION

All evidence is fused by IBM Granite in a single orchestration step. Granite reads the ML scores, drainage conditions, citizen reports, and agent analyses — all grounded in the live application data — and synthesizes a unified zone intelligence picture.

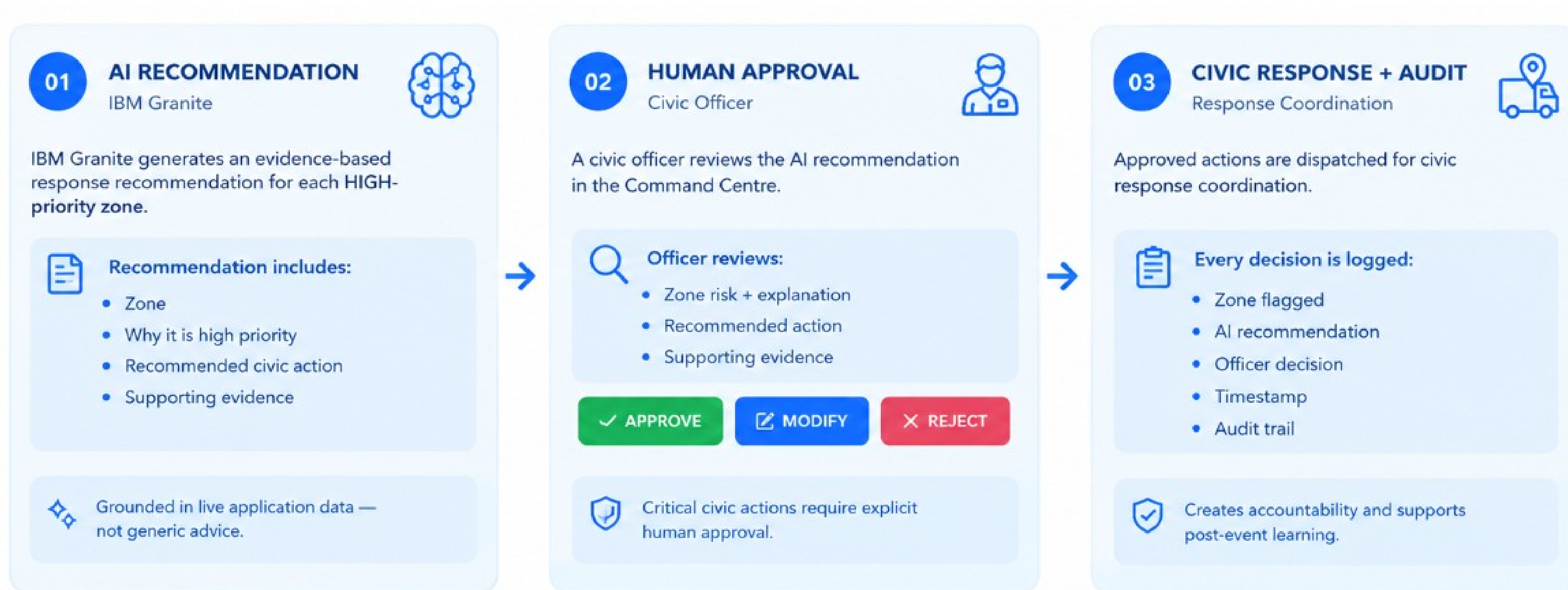
Output: Zone-level explanation + priority ranking + recommended action per zone.

ZONE PRIORITIZATION OUTPUT: HIGH RISK ZONES (immediate action) · MEDIUM RISK ZONES (monitoring + standby) · MONITOR (low priority, watch)

IBM Granite explains not only which zone is at risk, but why — based on the combined evidence.

AI Proposes. Humans Decide. Civic Actions Are Never Autonomous.

Every recommendation is evidence-based, reviewed by a civic officer, and logged for accountability.



FloodGuard AI supports decision-making. Critical actions remain subject to explicit



FLOODGUARD AI

AI FLOOD COMMAND CENTER — AHMEDABAD & SURAT, GUJARAT

18 Sep 2026 · 18:15 UTC

Agentic AI system combining ML flood-risk prediction, IBM Granite reasoning, citizen intelligence and drainage analytics for urban flood emergency management.

HYBRID DATA

6 AGENTS ACTIVE

IBM GRANITE

DEMO/SYNTHETIC



Citizen Portal

Report floods · Track status · Multilingual AI

EN · हिंदी · ગુજ

Open Citizen Portal



Command Center

Live map · Risk zones · Incidents · Teams

LIVE MAP

Open Command Center



Agent Monitor

Watch 6 agents · WHY explanations · NL query

AGENT TRACE

Open Agent Monitor



Analytics & ML

Trends · Feature importance · Model evaluation

RANDOM FOREST

Open Analytics & ML



Emergency War Room

Chief Agent · Approvals · Resources · HITL

HUMAN APPROVAL

Open Emergency War Room



Flood Simulator

What-if · Scenario sliders · Before/After

DRAINAGE SIM

Open Flood Simulator



Learning Loop

Prediction vs outcome · Accuracy tracking

CLOSED LOOP

Open Learning Loop



Decision Intelligence

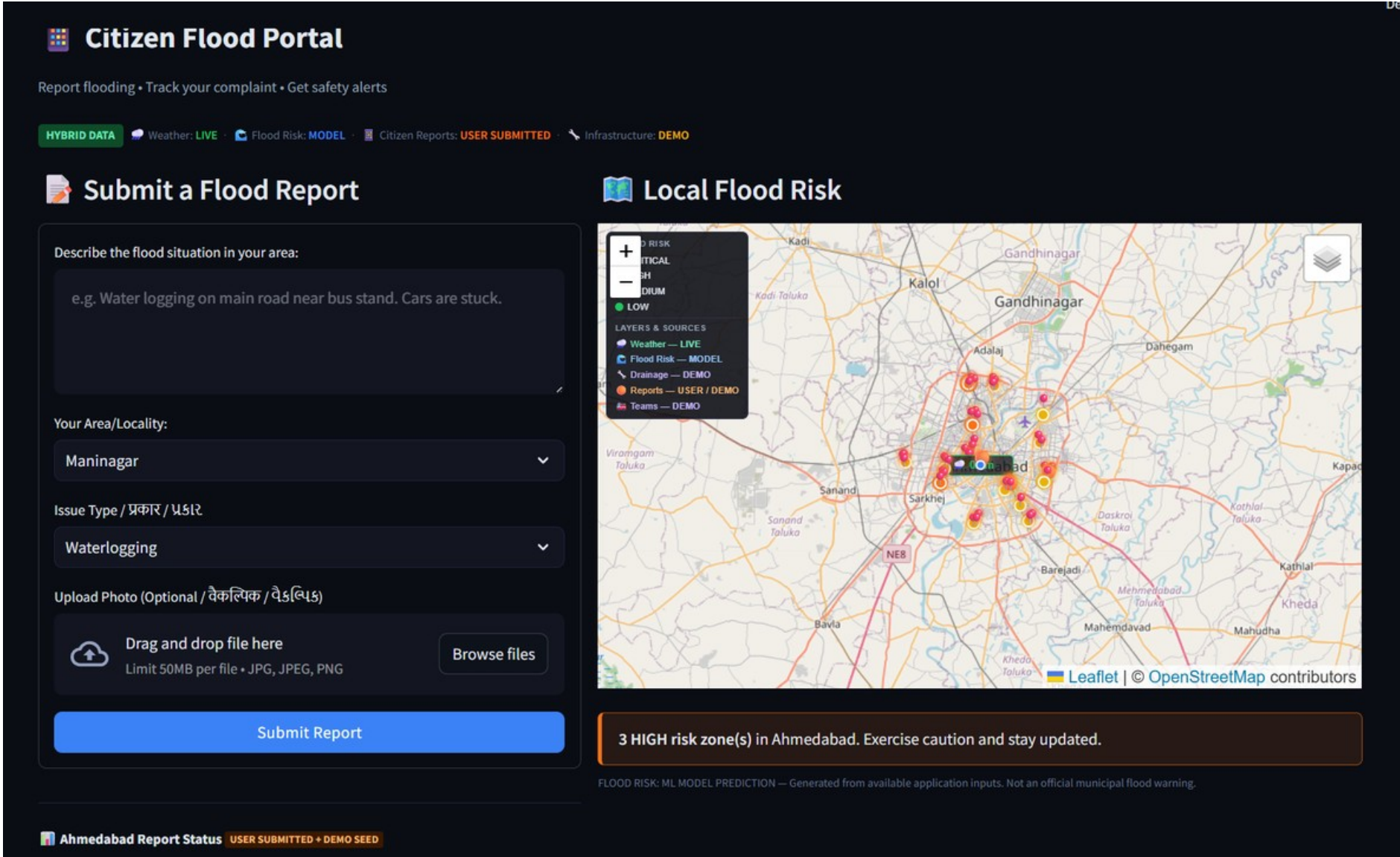
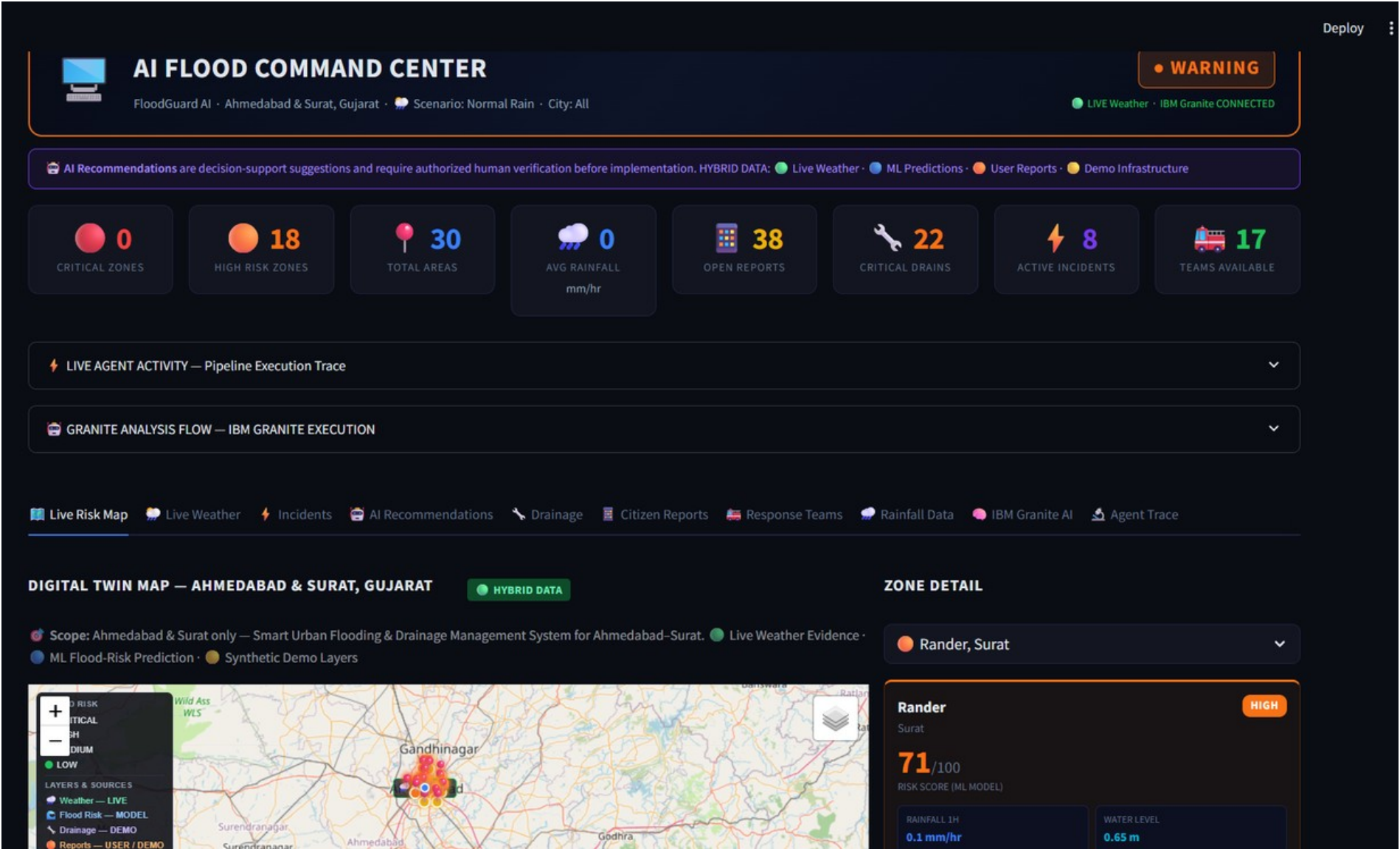
Evidence fusion · Priority · Why? · Audit

EXPLAINABILITY

Open Decision Intelligence

• APPLICATION MODULES

Command Centre • Citizen Portal

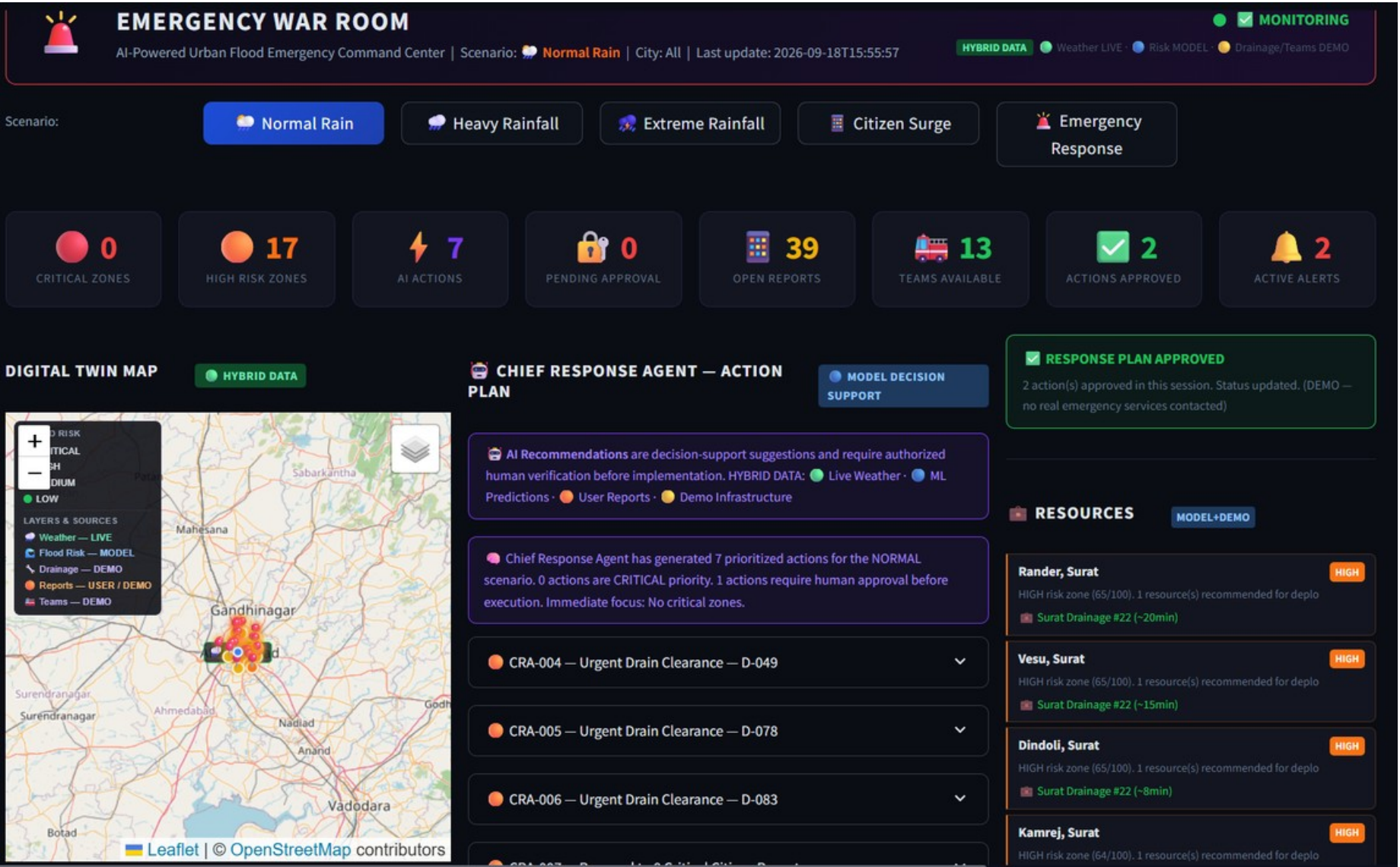


COMMAND CENTRE — Consolidated civic intelligence: geospatial map, AI recommendations, active incidents, human approval workflow

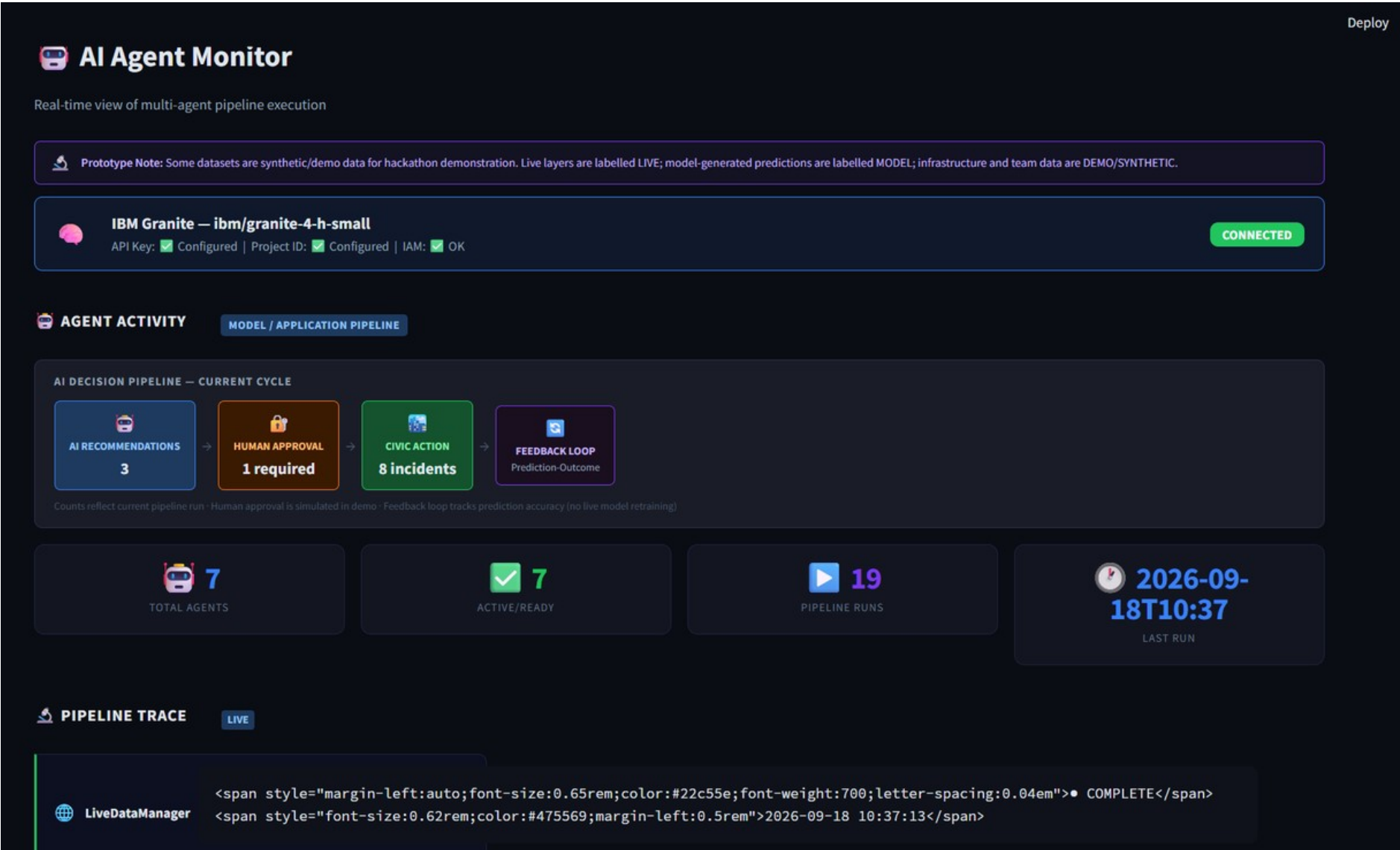
CITIZEN PORTAL — Crowd-sourced flood report submission: citizens contribute real-time ground observations that feed the Agent Analysis layer

• APPLICATION MODULES

War Room · Agent Monitor



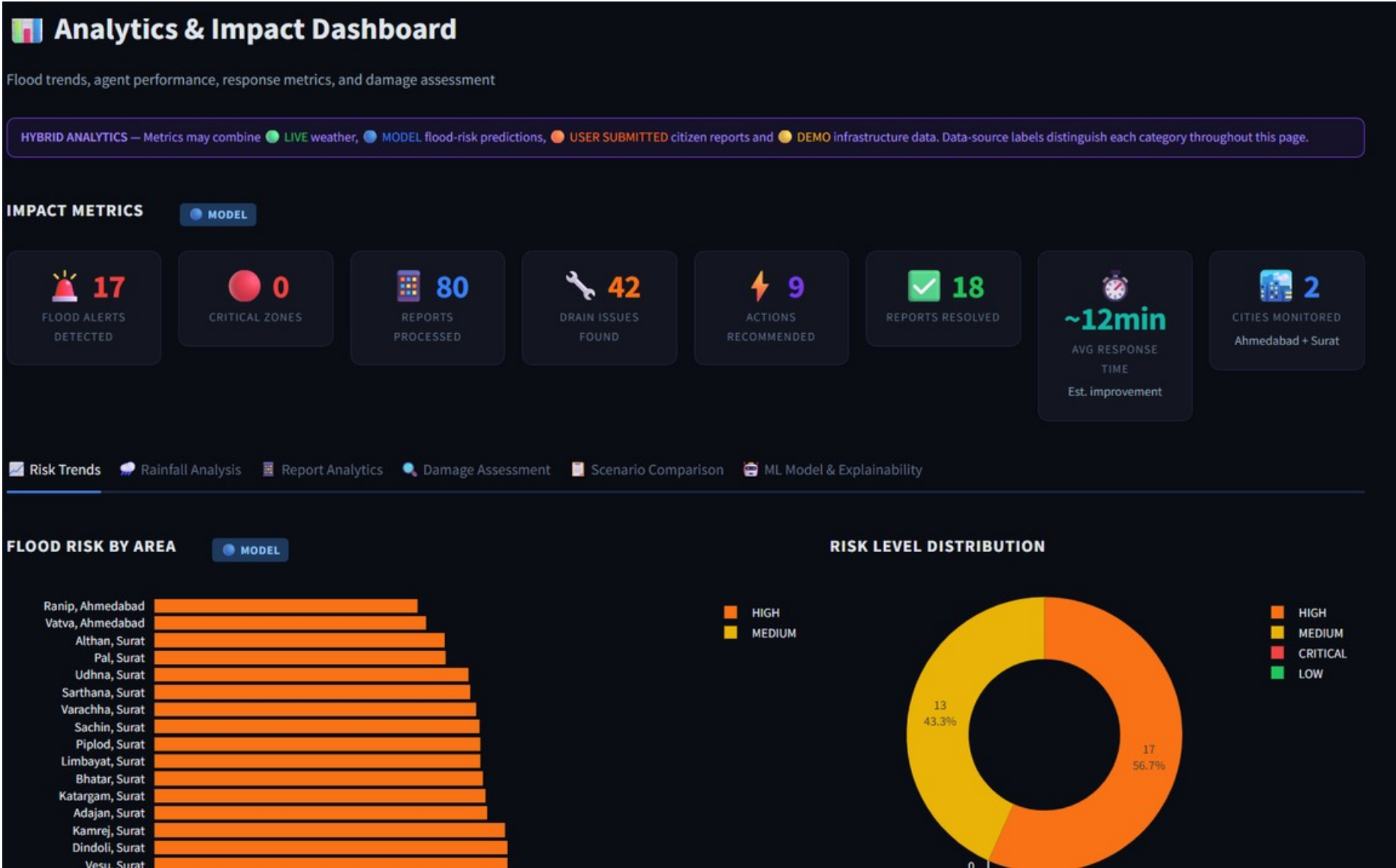
WAR ROOM — Response coordination centre: zone assignments, resource dispatch, team communication, and incident escalation in one view



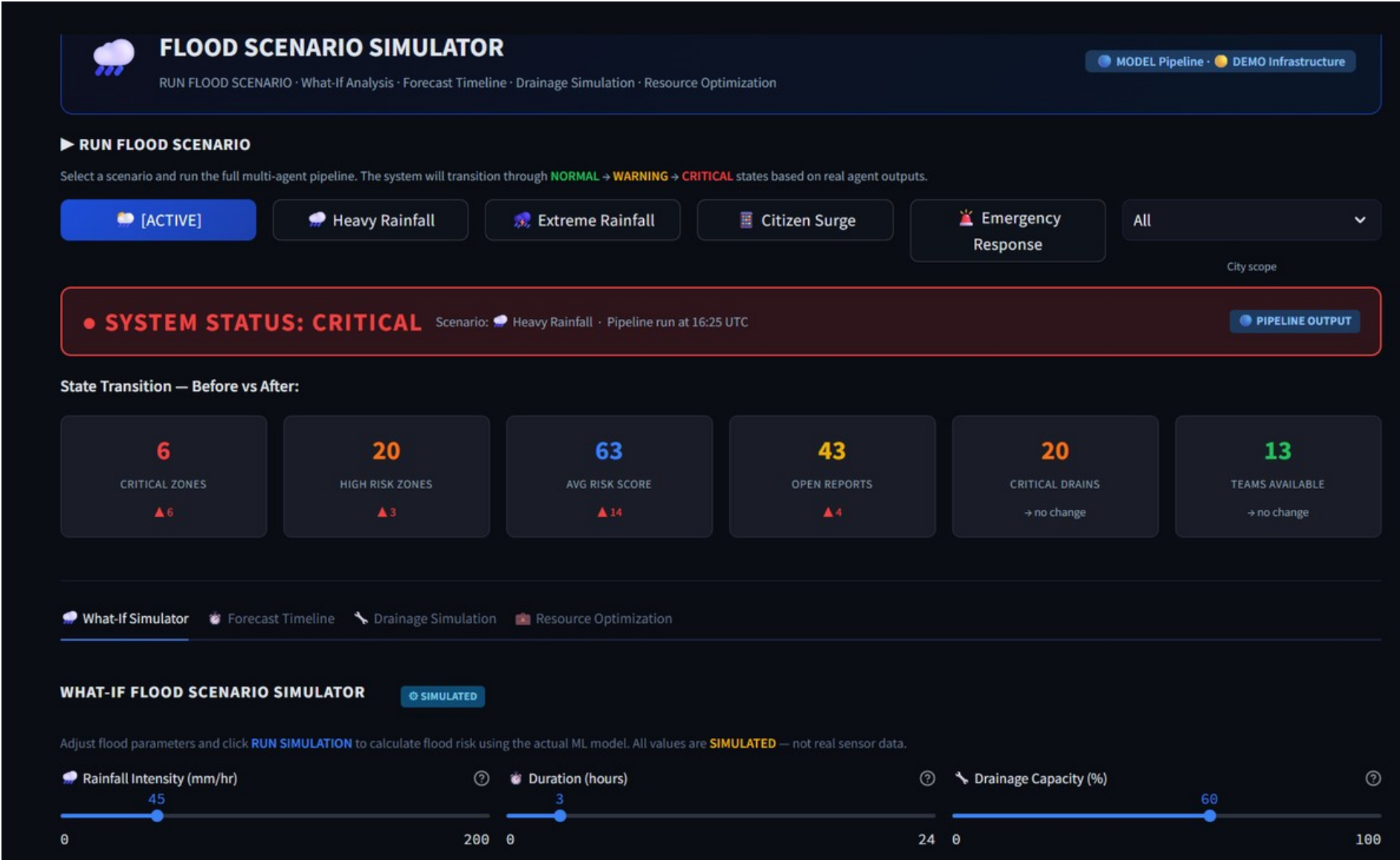
AGENT MONITOR — Live visibility into all four agents: what each agent is analyzing, current status, outputs, and confidence levels

• APPLICATION MODULES

Analytics · Simulator



ANALYTICS - Historical flood trends, zone risk patterns, drainage performance over time



SIMULATOR What-if scenario planning: test response strategies against different rainfall and blockage scenarios

- APPLICATION MODULES

Live Weather

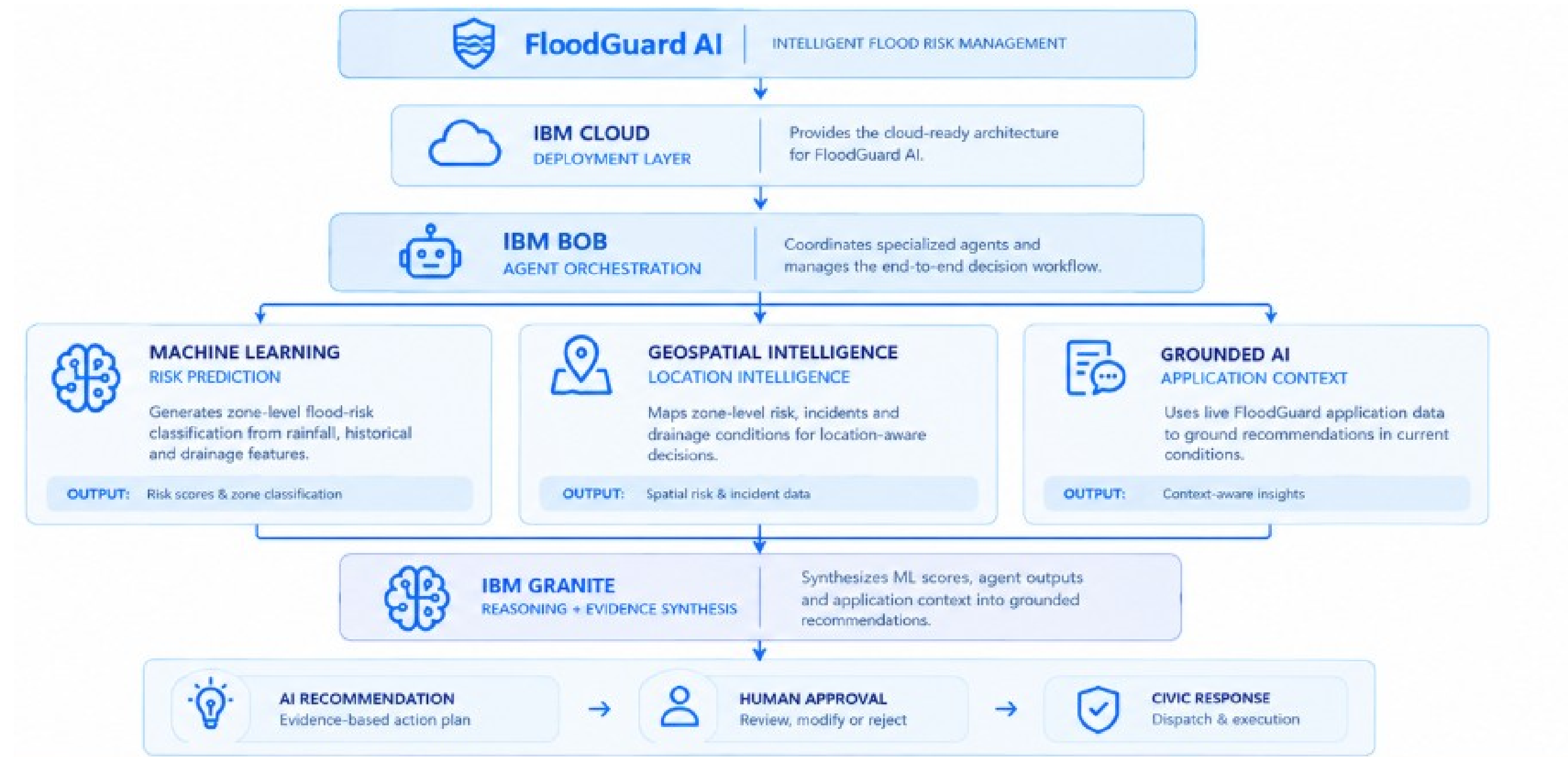


Real-time weather data
Rainfall • Wind • Humidity

LIVE WEATHER
Rainfall • Temperature • Humidity • Wind

IBM Technologies — How Each Contributes

Each IBM technology supports a specific layer of FloodGuard AI's decision-intelligence pipeline.



IBM technologies are not listed as logos here — they are structural components of the working pipeline.

What Makes FloodGuard AI Different

FloodGuard AI combines prediction, multi-agent intelligence, grounded reasoning and human oversight into one civic decision system.

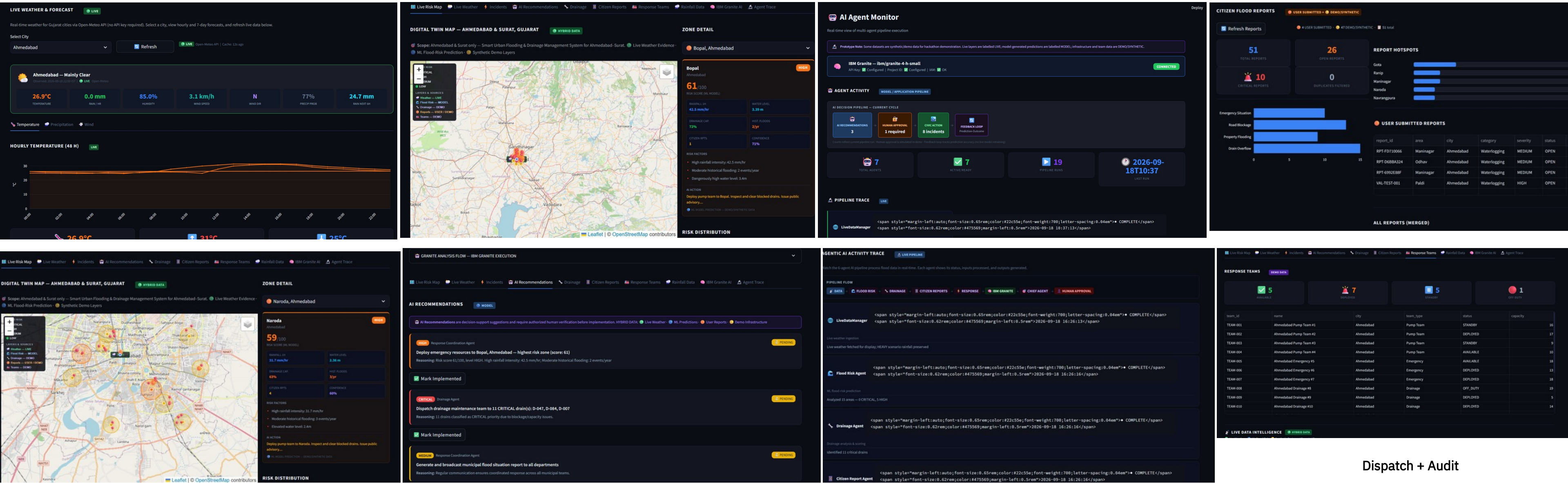


Purpose-built for civic flood intelligence — from prediction to explainable, human-approved action.

• END-TO-END DEMONSTRATION FLOW

The Demo — One Continuous Story

This is the working application. Every step shown here is implemented and demonstrable in the live FloodGuard AI system.



Dispatch + Audit

LIVE WEATHER → ML PREDICTION → MULTI-AGENT ANALYSIS → EVIDENCE FUSION → ZONE PRIORITIZATION → AI RECOMMENDATION → HUMAN APPROVAL → CIVIC RESPONSE → AUDIT LOG

What FloodGuard AI Enables — And Where It Can Go

IMMEDIATE IMPACT — AS IMPLEMENTED

- Faster Response — Consolidated intelligence dashboard
- Evidence-Based Prioritization — Resources directed to high-risk zones
- AI-Assisted Decisions — Explanations + recommendations for officers
- Citizen Participation — Flood reports as ground-level evidence
- Full Audit Trail — AI recommendations + human decisions logged
- Scenario Simulation — Monsoon preparedness planning

FUTURE SCOPE — POTENTIAL DIRECTIONS

- More Cities — Gujarat-wide municipal expansion
- Real Sensors — Rain gauges, drainage sensors, CCTV
- Citizen Alerts — Early warnings for high-risk zones
- Enhanced IBM AI — Deeper watsonx integration
- Multi-Hazard Intelligence — Heat, air quality, waterborne risks
- Continuous Learning — Post-event feedback improves ML
- Civic Infrastructure Platform — Regional adoption

- CONCLUSION

FLOODGUARD AI

From Fragmented Data to Coordinated Civic Response

DETECTION → ML PREDICTION → AGENT ANALYSIS → EVIDENCE FUSION → ZONE PRIORITIZATION → AI
RECOMMENDATION → HUMAN APPROVAL → CIVIC RESPONSE

01 — PREDICT

ML identifies flood risk by zone.

02 — UNDERSTAND

Agents + IBM Granite combine
evidence and generate explainable
recommendations.

03 — ACT

Human officers approve prioritized civic
actions with a complete audit trail.

“FloodGuard AI doesn't just predict flooding. It converts prediction + citizen reports + drainage
information into prioritized, human-approved civic actions.”

GitHub: github.com/PrinceJain2006/floodguard-ai

Prince Jain · Alka Raikwar · Aniket Jaiswal · IBM SkillsBuild Bob Hackathon · Gujarat 2026

BUILD WITH

IBM Bob — Agentic AI orchestration
IBM Granite — Reasoning & recommendations
IBM Cloud — Cloud-ready architecture
Machine Learning — Flood-risk prediction
Geospatial Intelligence — Risk mapping
Streamlit + Python — Application

FileEditSelectionViewGoRunTerminalHelp

←→floodguard

Upgrade your plan

EXPLORER

FLOODGUARD

.streamlit

agents

backend

models

config.py

data

frontend

logs

ml

pages

venv

etc\jupyter\nbconfig\notebook.d

jupyterlab-plotly.json

pydeck.json

Include

Lib

Scripts

share

pyvenv.cfg

.dockerignore

.env.example

.gitignore

app.py

ARCHITECTURE.md

DEMO_SCRIPT.md

Dockerfile

ibm-ce-config.yaml

ibm-ce-deploy.sh

README.md

OUTLINE

TIMELINE

VS CODE PETS

Bob Edit Preview of granite_service.py

C: > Users > Prince > .bob > playground > floodguard > agents > granite_service.py

26 def _get_iam_token() -> str | None:

33 try:

34 resp = httpx.post(

39+)

40+ resp.raise_for_status()

41+ token = resp.json()["access_token"]

42+ _iam_cache.update({"token": token, "expires_at": now + 55 * 60})

43+ return token

44+ except Exception as e:

45+ print(f"[Granite] IAM token error: {e}")

46+ return None

47+

48+

49+ def _call_granite(prompt: str, max_tokens: int = 600, temperature: float = 0.3) -> str | None:

50+ """Call Granite text generation via WatsonX REST API."""

51+ token = _get_iam_token()

52+ if not token:

53+ return None

54+ url = f"{WATSONX_URL}/ml/v1/text/generation?version=2023-05-29"

55+ payload = {

56+ "model_id": GRANITE_MODEL_ID,

57+ "project_id": WATSONX_PROJECT_ID,

58+ "input": prompt,

59+ "parameters": {

60+ "decoding_method": "greedy",

61+ "max_new_tokens": max_tokens,

62+ "temperature": temperature,

63+ "stop_sequences": ["<|endoftext|>"],

64+ },

65+ }

66+ try:

67+ resp = httpx.post(

68+ url,

69+ json=payload,

70+ headers={"Authorization": f"Bearer {token}", "Content-Type": "application/json"},

71+ timeout=30,

72+)

IBM BOB

Pasted text #1 176.7k / 270.0k 20.53

Rebuild agents/granite_servic... 4/

Mujhe sirf closed_loop_learning.py page ka UI aur code better banana hai, baaki sab waise hi rahega

Mujhe existing project ko ek fresh copy mein recreate karna hai (duplicate/backup banana hai)

Asked follow up >

Understand current project structure bef...

Listed files in . >

Stop the running Streamlit process

Ran 2 commands >

Tools awaiting approval:

Write File ... Open Dif

This operation accesses protected Bob files and requires explicit approval.

Approve once Reject

2 files changed Undo all Show all

Follow up, ↵ to steer, Alt ↵ to queue, esc to cancel

Agent Permissions

main* 0 0 Connect Bob Findings

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.12.7 (venv: venv) Go Live Bob Settings

EXPLORER

- > FLOODGUARD
- > OUTLINE
- > TIMELINE
- > VS CODE PETS

```
Bob Edit Preview of granite_service.py X
C: > Users > Prince > .bob > playground > floodguard > agents > granite_service.py

26 def _get_iam_token() -> str | None:
33     try:
34         resp = httpx.post(
39+         )
40+         resp.raise_for_status()
41+         token = resp.json()["access_token"]
42+         _iam_cache.update({"token": token, "expires_at": now + 55 * 60})
43+         return token
44+     except Exception as e:
45+         print(f"[Granite] IAM token error: {e}")
46+         return None
47+
48+
49+ def _call_granite(prompt: str, max_tokens: int = 600, temperature: float = 0.3) -> str | None:
50+     """Call Granite text generation via WatsonX REST API."""
51+     token = _get_iam_token()
52+     if not token:
53+         return None
54+     url = f"{WATSONX_URL}/ml/v1/text/generation?version=2023-05-29"
55+     payload = {
56+         "model_id": GRANITE_MODEL_ID,
57+         "project_id": WATSONX_PROJECT_ID,
58+         "input": prompt,
59+         "parameters": {
60+             "decoding_method": "greedy",
61+             "max_new_tokens": max_tokens,
62+             "temperature": temperature,
63+             "stop_sequences": ["<|endoftext|>"],
64+         },
65+     }
66+     try:
67+         resp = httpx.post(
68+             url,
69+             json=payload,
70+             headers={"Authorization": f"Bearer {token}", "Content-Type": "application/json"},
71+             timeout=30,
72+         )
```

IBM BOB

Pasted text #1 176.7k / 270.0k 20.53

Rebuild agents/granite_servic... 4/

Mujhe sirf closed_loop_learning.py page ka UI aur code better banana hai, baaki sab waise hi rahega

Mujhe existing project ko ek fresh copy mein recreate karna hai (duplicate/backup banana hai)

Asked follow up >

- Understand current project structure bef...

Listed files in . >

- Stop the running Streamlit process

Ran 2 commands v

Tools awaiting approval:

Write File ... Open Diff

This operation accesses protected Bob files and requires explicit approval.

Approve once Reject

2 files changed Undo all Show all

Follow up, ↵ to steer, Alt ↵ to queue, esc to cancel

+ Agent Permissions

FileEditSelectionViewGoRunTerminalHelp

←→floodguard

Upgrade your plan

🔍📄📁📂📌

IBM BOB

⏮⚙️🔍⏭⋮🔗

EXPLORER

▼ FLOODGUARD

> .streamlit

▼ agents

🔗 __init__.py

🔗 chief_response_agent.py

🔗 citizen_report_agent.py

🔗 closed_loop_learning.py

🔗 damage_assessment_agent.py

🔗 drainage_agent.py

🔗 flood_risk_agent.py

🔗 granite_service.py

🔗 orchestrator.py

🔗 response_coordination_agent.py

▼ backend

> models

🔗 __init__.py

🔗 config.py

> data

> frontend

▼ logs

> ml

> pages

▼ venv

▼ etc\jupyter\nbconfig\notebook.d

🔗 jupyterlab-plotly.json

🔗 pydeck.json

> Include

> Lib

> Scripts

> share

> OUTLINE

> TIMELINE

> VS CODE PETS

Bob Edit Preview of 5_war_room.py

C: > Users > Prince > .bob > playground > floodguard > pages > 5_war_room.py

36+
37+ with st.sidebar:
38+ st.markdown("## 🚨 Emergency War Room")
39+ st.markdown("---")
40+ st.markdown("### 📄 Activate Scenario")
41+ for key, info in SCENARIOS.items():
42+ if st.button(f"{info['emoji']} {info['label']}", key=f"wr_{key}", use_container_width=True):
43+ with st.spinner(f"Activating {info['label']}..."):
44+ orch.run_pipeline(key, "All")
45+ st.rerun()
46+ st.markdown("---")
47+ st.markdown(f'<div style="font-size:0.75rem;color:#94a3b8">{demo_badge()}</div>', unsafe_allow_html=True)
48+
49+ state = orch.current_state or {}
50+ preds = state.get("risk_predictions", [])
51+ critical = [p for p in preds if p["risk_level"] == "CRITICAL"]
52+
53+ header("Emergency War Room", "Live command · Chief Agent · Approvals · Resource allocation", "📄")
54+
55+ if critical:
56+ st.markdown(f"""
57+ <div style="background:rgba(239,68,68,0.15);border:2px solid #ef4444;border-radius:8px;
58+ padding:0.8rem 1rem;margin-bottom:1rem;text-align:center">
59+
60+ 🚨 {len(critical)} CRITICAL FLOOD ZONE(S) ACTIVE – EMERGENCY PROTOCOLS ENGAGED
61+
62+ <div style="font-size:0.75rem;color:#94a3b8;margin-top:0.2rem">🚧 SIMULATED – Not a real flood
63+ </div>
64+ """, unsafe_allow_html=True)
65+
66+ ai_disclaimer()
67+
68+ # — KPIs —
69+ action_plan = state.get("action_plan", {})
70+ actions = action_plan.get("actions", [])
71+ approval_n = action_plan.get("approval_needed", 0)
72+ approved_n = len(st.session_state.war_approved)

Pasted text #1 250.5k / 270.0k 25.69

🔍 Todo List 4/36

● Understand current project structure b...

● Stop the running Streamlit process

● Delete all existing application source fi...

● Rebuild app.py — Landing page

● Rebuild agents/granite_service.py — ...

○ Rebuild agents/orchestrator.py — Cent...

○ Rebuild agents/flood_risk_agent.py — ...

○ Rebuild agents/drainage_agent.py — ...

○ Rebuild agents/citizen_report_agent.p...

○ Rebuild agents/response_coordination...

○ Rebuild agents/chief_response_agent.p...

○ Rebuild agents/damage_assessment_a...

○ Rebuild agents/closed_loop_learning.p...

○ Rebuild agents/_init_.py

Tools awaiting approval:

📄 Write File ... Open Diff

⚠️ This operation accesses protected Bob files and requires explicit approval.

✓ Approve once ✕ Reject

📄 28 files changed Undo all Show all

Follow up, ⌘ to steer, Alt ⌘ to queue, esc to cancel

+ Agent ▼ ⚙️ Permissions ▼

main* 0 0 Connect Bob Findings

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.12.7 ('venv': venv) Go Live Bob Settings

LIVE FLOODGUARD DASHBOARD



FLOODGUARD AI

AI FLOOD COMMAND CENTER — AHMEDABAD & SURAT, GUJARAT

18 Sep 2026 · 18:1

HYBRID DATA

6 AGENTS ACTIVE

IBM GRANITE

DEMO/SYNTH

Report floods · Track status · Multilingual AI

EN · ગિદી · ગુ

Live map · Risk zones · Incidents · Teams

LIVE MAP

Watch 6 agents · WHY explanations · NL query

AGENT TRACE

Trends · Feature importance · Model evalua

RANDOM FOREST

Open Citizen Portal

Open Command Center

Open Agent Monitor

Open Analytics & ML

Emergency War Room

Chief Agent · Approvals · Resources · HITL

HUMAN APPROVAL

Open Emergency War Room

Flood Simulator

What-if · Scenario sliders · Before/After

DRAINAGE SIM

Open Flood Simulator

Learning Loop

Prediction vs outcome · Accuracy tracking

CLOSED LOOP

Open Learning Loop

Decision Intelligence

Evidence fusion · Priority · Why? · Audit

EXPLAINABILITY

Open Decision Intelligence

6-AGENT AI ARCHITECTURE

Multi-agent pipeline with IBM Granite reasoning layer

DATA SOURCES

Flood Risk Agent

Drainage Agent

Citizen Report Agent

Response Agent

IBM GRANITE

Chief Response Agent

HUMAN APPROVAL

RESPONSE

Flood Risk Agent

ML flood prediction per area

Drainage Agent

Drain maintenance prioritization

Citizen Report Agent

Multilingual NLP classification

Response Coord Agent

Incident response planning

Chief Response Agent

Unified emergency action plan

Damage Agent

Post-flood damage assessment

Learning Loop

Prediction+outcome feedback

Decision Intelligence

Evidence fusion + explainability

IBM Granite Integration

ibm/granite-4-b-small

Provides situation summaries · Incident classification · multilingual report understanding · recommendation explanations · natural language Q&A via WatsonX API. Set WATSONX_API_KEY + WATSONX_PROJECT_ID to enable live mode. Falls back gracefully to rule-based responses with clear GRANITE FALLBACK labelling.

Configure in .env

Fallback always active

DATA TRANSPARENCY

FloodGuard AI combines live weather (Open-Meteo API), ML flood-risk predictions (Random Forest), IBM Granite AI reasoning, user-submitted citizen reports, and clearly labelled DEMO/SYNTHETIC infrastructure data. Drainage assets, response teams and baseline flood incidents are demo data. All AI recommendations require authorized human verification before real-world implementation. This application is a hackathon demonstration and does not constitute an official municipal emergency system.

Thank You!

Thank you for your time and interest.